Assignment 2: Due April 26th, 2026

Turn-in

This is an individual assignment. You are required to submit both the answers and code you used for this assignment on Gradescope, instructions on how to join the course on Gradescope and submit the assignment are posted at the end of this writeup. You may submit a PDF writeup along with code for your submission, or submit a jupyter notebook with the code + answers. Remember to fully evaluate the cells in your jupyter notebook if you choose that option for submission. Late submissions will be accepted and deducted in accordance to the course late-day policy.

Overview

In this assignment, you will extract Quality-of-Service (QoS) metrics for a YouTube video streaming application from a given packet capture. You are given a PCAP file in which the host (with IP address `192.168.1.103`) is watching a YouTube video.

The goal of this assignment is to give you practice programatically analyzing packet captures. You will investigate how quality of service metrics are calculated for multimedia video streaming applications to give you an idea about how these applications make real-time decisions such as what bitrate or resolution to use when delivering you content.

Getting Started

First download the packet capture: video streaming.pcap. This pcap was captured on an end host with IP address `192.168.1.103`, that is currently watching a YouTube video.

Before you begin analysis, you will need to convert the pcap file into a CSV format. Working directly with the pcap binary format requires specialized libraries and more complex code, whereas a CSV gives us a simple, tabular representation of each packet's fields that we can easily load and manipulate with standard tools like Python's `csv` module or `pandas`. Note that CSV is not even the most optimal format for processing large amounts of network data; formats like Parquet offer better performance and compression, but CSV provides ease-of-use since the data is stored as human-readable text that you can inspect and debug at a glance.

To extract fields from a pcap into a CSV, you can use `tshark`, the command-line version of Wireshark. The `-T fields` option tells tshark to output specific fields, and you use `-e` to specify each field you want to extract. For example:

```
tshark -r video_streaming.pcap -T fields -E separator=, -E header=y \
  -e frame.number -e frame.time_epoch -e ip.src -e ip.dst \
  -e ip.len -e dns.qry.name -e dns.a \
  > video_streaming.csv
```

Some useful tshark field names include:

- `frame.number` — the packet number in the capture
- `frame.time_epoch` — the timestamp of the packet (in seconds since epoch)
- `ip.src` / `ip.dst` — source and destination IP addresses

- `ip.len` — the total IP packet length in bytes
- `dns.qry.name` — the DNS query name (useful for identifying which domains are being resolved)
- `dns.a` — the DNS A record response (the resolved IP address)

You may need to revisit this tshark command and add new fields as you progress through the assignment and want to process more of the data. For instance, you might later in tihs assignment need TCP or UDP port fields or additional protocol-specific fields depending on the questions you are answering.

You might wonder: why not just extract *all* fields from the pcap into the CSV at once, rather than iteratively adding fields as needed? The downside is the explosion in data size—a pcap with all possible fields extracted can produce a CSV that is significantly larger than the original capture. Reading and processing this bloated CSV will take more time and memory, which defeats the purpose of converting to a simpler format in the first place. By selectively extracting only the fields you need, you keep the CSV small relative to the original pcap and your analysis stays fast and manageable.

You may use any preferred language to programatically answer the following questions. The recommended workflow is to use the `pandas` library in Python for writing queries as we will have gone over in discussion section.

Isolating Relevant Packets

1. Find the IP address for the server(s) responsible for YouTube streaming traffic to the end host using the DNS responses. * Note that, the shared packet capture is collected from a production satellite network. To save disk space, it's truncated and therefore, we can't see the TLS handshake headers. Hence, we have to rely on DNS responses alone to isolate YouTube streaming traffic.

2. Display the top five source IP, destination IP pairs and sorted based on the number of bytes transferred **FROM** the source **TO** the destination.

3. Isolate the flow (identified by the 5-tuple) which sends the most amount of data to the host (`192.168.1.103`).

Chunk Detection

You may recall from CS 176A that video streaming applications like YouTube use **adaptive bitrate (ABR) streaming**. Rather than downloading an entire video file at once, the video player breaks the content into small segments called **chunks** (typically a few seconds of video each). The player issues HTTP GET requests to download these chunks one at a time, and after each chunk is received, it decides what bitrate or resolution to request for the next chunk based on current network conditions such as available bandwidth and buffer health. This is what allows a video to start playing quickly and adapt its quality in real time—you may have noticed a YouTube video starting at a lower resolution and then improving, or dropping in quality when your network slows down.

In an ideal analysis scenario, we would inspect the HTTPS headers directly to see each GET request and its corresponding response. Wireshark can decrypt TLS traffic if it has access to the session keys and can observe the full TLS handshake. However, this packet capture is truncated because it was collected on a production satellite network where, to save disk space, packet payloads were clipped. This means we cannot see the full TLS handshake or the

encrypted HTTP headers. This is also a common limitation when working with shared packet captures due to data privacy concerns; the payload data may contain sensitive user information. You are always welcome to capture your own packets to further explore what the application-layer headers would look like! You will get more experience doing this during the later programming assignments and the course project.

Since we cannot view the GET requests directly, we need to implement a heuristic-based algorithm that infers chunk boundaries by looking at packet sizes and directions. We will use an approach developed by **Requet** (Gutterman et al., *"Requet: Real-Time QoE Detection for Encrypted YouTube Traffic"*, MMSys '19). The algorithm works as follows:

- **Identify GET requests using packet size thresholds.** Since we cannot dissect the HTTP data due to truncation, we use a size-based heuristic: packets sent from the client to the server that exceed a certain threshold (`GET_thresh`) are assumed to be GET requests for video chunks. We also filter for relevant video packets using a `VIDEO_thresh` to exclude small control packets.

- **Associate response data with each GET request.** We make a simplified assumption that, on this constrained satellite network, the video player requests chunks sequentially. Therefore, all packets sent from the server to the client between two consecutive GET requests are carrying response data for the previously sent GET request. Packets exceeding `DOWN_thresh` in the server-to-client direction are counted as download data for the current chunk.

- **Compute per-chunk metrics.** Once chunk boundaries are identified, we can compute QoS metrics for each chunk.

Using the algorithm description above, try and implement the chunk detection algorithm. You may use the following threshold values for your algorithm, but you may optionally experiment with different values if you are curious how the output of your algorithm changes:

- `GET_thresh = 300` (bytes)
- `DOWN_thresh = 300` (bytes)
- `VIDEO_thresh = 800` (bytes)

4. How many chunks do you observe being requested in the packet capture? What threshold did you use?

5. Plot the size of each chunk across time. The X-axis should be time (e.g., the relative timestamp of the GET request), and the Y-axis should be the size of each chunk. Compute the chunk size as the sum of the IP payload length (`ip.len`) for all packets in the response data associated with each GET request.

6. Comment on how your graph turned out. How did you tune the threshold parameter used for identifying GET requests—what values gave you a graph that looked reasonable? Try to reason about any inconsistencies you observe in your graph, if any, or describe whether you had to do anything else (e.g., filter out additional data to produce a cleaner graph).

QoS Metrics

For each chunk you identified in the previous part, compute the following metrics, and graph them across time, similar to Question 6.

7. TTFB (Time to First Byte) (in seconds) — The time elapsed between when the client sends the GET request and when the first packet of response data arrives from the server. This measures how long the client waits before it begins receiving any data for the chunk.

8. Download Time (in seconds) — The time elapsed between the arrival of the first response packet and the arrival of the last response packet for the chunk. This measures how long it takes to fully download the chunk once data starts arriving.

9. Download Speed (in Kbps) — The rate at which the chunk was downloaded, computed as the total chunk size (in kilobits) divided by the download time (in seconds).

Gradescope

Submissions will made done through Gradescope as a PDF. You can enroll in the course through Gradescope using the entry code **4DWRD5**. Please use either your @ucsb.edu or @umail.ucsb.edu email when joining the course. You may submit a PDF writeup along with code for your submission, or submit a jupyter notebook with the code + answers. Remember to fully evaluate the cells in your jupyter notebook if you choose that option for submission. Late submissions will be accepted and deducted in accordance to the course late-day policy.